

CC3092 - Deep Learning y Sistemas Inteligentes

Laboratorio #7

NLP end-to-end y Embeddings

Instrucciones generales

- En parejas o individual.
- Entrega: lunes 05 de octubre, 2026. 23:59.

1. Datos

Trabjarán con dos conjuntos de datos públicos en inglés: un corpus de texto crudo para entrenar sus propios embeddings (WikiText-103, artículos de Wikipedia) y un dataset de clasificación de noticias en 4 categorías (AG News) para usar los embeddings en una tarea real. Como modelo preentrenado de referencia usarán GloVe de 100 dimensiones, entrenado sobre Wikipedia + Gigaword (6 mil millones de tokens), un dominio similar al de su corpus.

Corpus, dataset y modelo preentrenado:

```
from datasets import load_dataset
import gensim.downloader as api
corpus = load_dataset("Salesforce/wikitext", "wikitext-103-raw-v1")
news = load_dataset("fancyzhx/ag_news")
glove = api.load("glove-wiki-gigaword-100")
```

Benchmarks de evaluación (incluidos en gensim):

```
from gensim.test.utils import datapath
analogias = datapath("questions-words.txt") # 19,544 analogías, 14 categorías
wordsim = datapath("wordsim353.tsv")
simlex = datapath("simlex999.txt")
```

2. Exploración y preprocesamiento del texto

Construyan las primeras etapas del pipeline (texto crudo → tokens → IDs) y respondan:

- ¿Cuántos artículos, líneas y tokens tiene el corpus?
- Definan y justifiquen su normalización: minúsculas, puntuación, números y tokens especiales. ¿Qué decisiones deben ser consistentes con el vocabulario de GloVe para que la comparación sea justa?
- Implementen un tokenizador por palabra y compárenlo con uno de librería (spaCy o NLTK) en al menos 10 oraciones de ejemplo. ¿En qué casos difieren (contracciones, guiones, abreviaturas)?
- Grafiquen la frecuencia de las palabras contra su rango en escala log-log. ¿Se cumple la ley de Zipf? ¿Qué porcentaje de los tokens cubren las 10, 1,000 y 30,000 palabras más frecuentes?
- Construyan el vocabulario con un umbral de frecuencia mínima (min_count). ¿Qué tamaño de vocabulario y qué porcentaje de tokens desconocidos obtienen con al menos tres umbrales distintos?

- Implementen el submuestreo (subsampling) de palabras frecuentes de Word2Vec. ¿Qué porcentaje de las apariciones de "the", "of" y "and" se descarta? ¿Por qué ayuda al entrenamiento?

Finalmente, generen los pares (palabra central, contexto) para skip-gram con una ventana de contexto y reporten cuántos pares de entrenamiento produce el corpus por epoch, antes y después del submuestreo.

3. Investigación: embeddings en PyTorch y gensim

Investiguen las clases, funciones y herramientas necesarias para entrenar, cargar y evaluar embeddings. Para cada una, describan brevemente su propósito y sus parámetros más relevantes.

- `nn.Embedding` (`num_embeddings`, `embedding_dim`, `padding_idx`, `sparse`) y `nn.Embedding.from_pretrained` (`freeze`)
- `nn.EmbeddingBag` y sus modos de agregación (`mean`, `sum`, `max`) usando offsets

Adicionalmente, investiguen brevemente los siguientes conceptos: diferencia entre CBOW y skip-gram; por qué Word2Vec usa dos matrices (W de entrada y W' de salida) y cuál se conserva; diferencia entre Word2Vec (predictivo) y GloVe (basado en conteos de co-ocurrencia).

4. Construcción y entrenamiento de los embeddings

Obtengan tres conjuntos de embeddings de palabras:

- **Skip-gram desde cero (PyTorch):** implementación propia de skip-gram con negative sampling (SGNS), con dos tablas `nn.Embedding` (entrada y salida), entrenada sobre su corpus preprocesado. Para este modelo no se permite usar la clase `Word2Vec` de `gensim`.
- **Word2Vec de gensim:** skip-gram entrenado sobre exactamente el mismo corpus, vocabulario e hiperparámetros equivalentes a su mejor configuración. Sirve como implementación de referencia para validar la suya.
- **GloVe preentrenado:** `glove-wiki-gigaword-100`, usado tal cual, sin entrenamiento adicional.

Pueden usar un subconjunto de WikiText-103 (mínimo 20 millones de tokens) si tienen restricciones de cómputo, siempre que sea el mismo para ambos modelos entrenados. Al menos una configuración debe tener dimensión 100 para compararla directamente con GloVe.

Al menos una de las iteraciones debe variar la dimensión del embedding (por ejemplo, 50, 100 y 300) y otra el tamaño del corpus (por ejemplo, 25 %, 50 % y 100 % del subconjunto). Otros hiperparámetros a explorar: tamaño de ventana, número de negativos, umbral de submuestreo, `min_count`, `learning rate` y `epochs`.

Para la selección de configuraciones usen únicamente WordSim-353 y las analogías de `questions-words.txt`. SimLex-999 se reserva para la evaluación final (sección 6).

Para cada iteración, registren:

- La configuración de hiperparámetros usada, incluyendo el número de tokens del corpus.
- La pérdida de entrenamiento por epoch (o cada N pasos).
- Al final de cada epoch: accuracy de analogías (semánticas, sintácticas y total) y correlación de Spearman en WordSim-353.
- Al final de cada epoch: las 5 palabras más cercanas a un conjunto fijo de palabras de control (por ejemplo: king, france, computer, good, january, run).
- El tiempo de entrenamiento por epoch y el tiempo total, y la memoria pico de GPU.

Grafiquen las curvas de pérdida y de accuracy de analogías por epoch de al menos 3 iteraciones.

¿Una pérdida más baja implica siempre mejores embeddings?

5. Verificación de la aritmética vectorial

Con su mejor modelo SGNS, el modelo de gensim y GloVe, verifiquen si se cumple la aritmética vectorial de los embeddings, por ejemplo $\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$.

5.1 Analogías individuales

- Implementen su propia función analogía(a, b, c, k) que resuelva «a es a b como c es a ?» con 3CosAdd, usando similitud coseno sobre vectores normalizados y excluyendo a, b y c del resultado. Verifiquen que coincide con most_similar de gensim.
- Evalúen al menos 15 analogías propias de al menos 5 tipos distintos (género, país–capital, país–gentilicio, comparativos y superlativos, tiempos verbales, plurales, etc.). Para cada modelo reporten el top-5 con sus similitudes, el rango de la respuesta correcta y su similitud coseno con el vector resultante.
- Repitan las analogías sin excluir las palabras de la consulta. ¿Qué palabra aparece en primer lugar? ¿Qué dice esto sobre qué tan exacta es la aritmética?

5.2 Evaluación sistemática

- Evalúen los tres modelos sobre questions-words.txt usando un vocabulario compartido (las 30,000 palabras más frecuentes presentes en los tres modelos), para que las preguntas evaluadas sean las mismas. Reporten la cobertura (preguntas evaluadas / total) y el accuracy por cada una de las 14 categorías con 3CosAdd y con 3CosMul.
- Visualicen con t-SNE alrededor de 500 palabras de grupos temáticos (países, números, animales, verbos, etc.) para su mejor modelo. ¿Se forman los grupos esperados?

6. Pipeline end-to-end: clasificación de texto

Para cerrar el pipeline (texto → tokens → vectores → modelo → salida), usen los embeddings en la clasificación de AG News, aplicando el mismo preprocesamiento y tokenizador de la sección 2. Separen 10 % del conjunto de entrenamiento como validación (estratificado) y reserven el conjunto de test oficial para la evaluación final. Reporten el porcentaje de tokens de AG News fuera del vocabulario de cada conjunto de embeddings.

Entrenen y comparen los siguientes modelos:

- **Baseline:** TF-IDF + regresión logística.
- **Embeddings aleatorios:** nn.EmbeddingBag (promedio) + MLP, con la tabla de embeddings inicializada aleatoriamente y entrenada con la tarea.
- **Su SGNS y GloVe:** el mismo clasificador inicializado con cada conjunto de embeddings, en dos variantes: (a) tabla congelada y (b) tabla con fine-tuning.

Para cada modelo registren accuracy, precision, recall y F1-score (macro) en validación, curvas de pérdida de entrenamiento y validación, parámetros totales y entrenables, y tiempo de entrenamiento. Evalúen la mejor variante de cada inicialización una única vez sobre test y generen su matriz de confusión. Reporten también la correlación de Spearman en SimLex-999 de los tres conjuntos de embeddings.

7. Comparación de resultados

Construyan una tabla comparativa entre su SGNS, Word2Vec de gensim y GloVe que incluya:

- Tokens del corpus de entrenamiento, tamaño del vocabulario y dimensión.
- Tiempo de entrenamiento y hardware utilizado (para GloVe, lo reportado por sus autores).
- Accuracy de analogías (semánticas, sintácticas y total) con 3CosAdd y 3CosMul.
- Correlación de Spearman en WordSim-353 y SimLex-999.
- Similitud coseno promedio entre vectores diferencia (sección 5.3).
- Porcentaje de tokens fuera del vocabulario en AG News y F1-score de test.

Adicionalmente, generen al menos dos gráficas: (1) accuracy de analogías contra número de tokens del corpus para sus iteraciones, con GloVe como línea de referencia; y (2) F1-score de test contra fracción de datos de entrenamiento de AG News, con una curva por tipo de inicialización.

8. Discusión y análisis

Respondan con base en sus resultados:

- ¿Se cumple la aritmética vectorial en su modelo? ¿En qué categorías de analogías funciona mejor y en cuáles falla? ¿Hay diferencias entre analogías semánticas y sintácticas?
- ¿Es el « $\approx$ » de $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ una igualdad? Usen los resultados sin exclusión de palabras de consulta y el análisis de paralelismo para argumentar.
- ¿Qué tan lejos quedó su modelo de GloVe? ¿Cuánto de la diferencia se explica por el tamaño del corpus y cuánto por el método o los hiperparámetros? Usen el experimento de tamaño de corpus para responder.
- ¿Su implementación de SGNS se comporta parecido a la de gensim? Si hay diferencias, ¿a qué se deben?

- ¿Qué efecto tuvieron la dimensión del embedding, el tamaño de ventana y el número de negativos? ¿Las ventanas pequeñas favorecen relaciones sintácticas o semánticas?
- En AG News, ¿los embeddings preentrenados superaron a los aleatorios y al baseline TF-IDF? ¿Cuándo convino congelar y cuándo hacer fine-tuning? ¿Cómo cambia la respuesta con 1 % de los datos?

Entregables

Entregable	Contenido
PDF (máx. 5 páginas)	Investigación, decisiones del pipeline de preprocesamiento, tabla de iteraciones del modelo skip-gram, resultados de la verificación de aritmética vectorial (analogías individuales, benchmark por categoría y paralelismo), tabla comparativa entre los tres conjuntos de embeddings, resultados de clasificación, análisis y conclusiones.
Repositorio (Git)	Jupyter Notebook completo y comentado: preprocesamiento y tokenización, implementación de skip-gram con negative sampling, entrenamiento de las iteraciones, evaluación de analogías y similitud, visualizaciones, clasificación con AG News y evaluación final sobre test. Incluyan los vectores de su mejor modelo (formato word2vec .txt o .kv) o un enlace para descargarlos.

Incluyan el enlace al repositorio al final del PDF.